

Stacks and Queues ADT

Data Structures Notes: Stacks and Queues

1. Introduction to Stack

A **Stack** is a linear data structure in which insertion and deletion of elements take place only at **one end**, called the **TOP**.

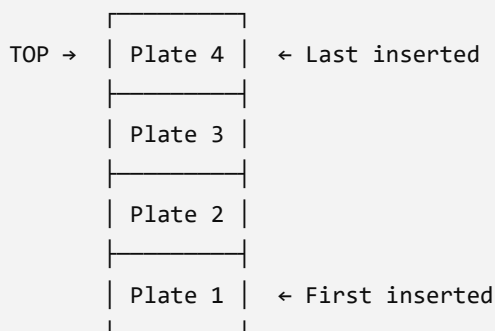
A stack follows the principle:

LIFO — Last In, First Out

The element inserted last is the first element removed.

Real-life example

Think of a stack of plates:



If Plate 4 is removed, it must be removed first.

Basic Stack terminology

- **TOP** – points to the topmost element.
- **Push** – insertion of an element.
- **Pop** – deletion of an element.
- **Peek/Top** – returns the top element without deleting it.
- **Overflow** – occurs when trying to push into a full stack.
- **Underflow** – occurs when trying to pop from an empty stack.

2. Stack as an Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** defines:

1. What data is stored.
2. What operations can be performed.
3. The behavior of those operations.

It does **not** specify how the data structure must be implemented.

A Stack ADT can be represented as:

```
Stack
├─ push(x)
├─ pop()
├─ peek()
├─ isEmpty()
└─ isFull()
```



A stack can be implemented using:

- Arrays
- Linked Lists

3. Stack Operations

3.1 Push Operation

Push inserts a new element at the top of the stack.

Suppose:

```
Stack = [10, 20, 30]
TOP = 30
```



After:

```
PUSH(40)
```



we get:

```
[10, 20, 30, 40]
      ↑
    TOP
```



Push Algorithm — Array Implementation

Assume:

- `STACK[MAX]` is the array.
- `TOP = -1` initially.

```
PUSH(STACK, ITEM)
```



```
1. If TOP == MAX - 1
    Print "Stack Overflow"
    Return
```

2. $TOP = TOP + 1$
3. $STACK[TOP] = ITEM$
4. Return

Complexity

Operation	Time Complexity
Push	$O(1)$
Space for one element	$O(1)$

Push requires only one position update and one assignment.

4. Pop Operation

Pop removes and returns the top element.

Example:

Before:

```
[10, 20, 30, 40]
      ↑
    TOP
```

After `POP()` :

```
[10, 20, 30]
      ↑
    TOP
```

The returned value is:

40

Pop Algorithm

`POP(STACK)`

1. If $TOP == -1$
 Print "Stack Underflow"
 Return
2. $ITEM = STACK[TOP]$

```
3. TOP = TOP - 1
```

```
4. Return ITEM
```

Complexity

Time Complexity = $O(1)$

Only the TOP position is accessed.

5. Peek / Top Operation

The **Peek** operation returns the top element without removing it.

Example:

Stack:

[10, 20, 30, 40]

↑

TOP



PEEK() returns:

40



The stack remains unchanged.

Algorithm

PEEK(STACK)

1. If TOP == -1
 Print "Stack is Empty"
 Return
2. Return STACK[TOP]



Complexity

Time Complexity = $O(1)$

6. isEmpty Operation

Checks whether the stack contains any elements.

Algorithm

```
isEmpty()
```



```
1. If TOP == -1
    Return TRUE
2. Else
    Return FALSE
```

Complexity

Time Complexity = $O(1)$

7. isFull Operation

For an array-based stack, checks whether the stack has reached its maximum capacity.

```
isFull()
```



```
1. If TOP == MAX - 1
    Return TRUE
2. Else
    Return FALSE
```

Complexity

Time Complexity = $O(1)$

8. Complexity of Stack Operations

Operation	Description	Time
Push	Insert element	$O(1)$
Pop	Delete top element	$O(1)$
Peek	Access top element	$O(1)$
isEmpty	Check empty	$O(1)$
isFull	Check full	$O(1)$

Therefore, basic stack operations are generally **constant time, $O(1)$** .

9. Applications of Stacks

Stacks are widely used in computer science.

Important applications include:

1. Function calls and recursion
2. Undo/Redo operations
3. Browser back button
4. Parentheses matching
5. Expression conversion
6. Expression evaluation
7. Backtracking
8. Depth First Search (DFS)
9. Syntax parsing in compilers
10. Memory management

The most important applications for this syllabus are:

Expression Conversion and Expression Evaluation

10. Expressions

An arithmetic expression consists of:

- **Operands** – variables or values
- **Operators** – `+`, `-`, `*`, `/`, etc.

For example:

`A + B`



Here:

- `A`, `B` → operands
- `+` → operator

There are three common forms of expressions.

10.1 Infix

Operator is written between operands.

`A + B`



Example:

`A + B * C`



This is the form commonly used by humans.

10.2 Prefix

Operator is written before operands.

+ A B



For:

A + B * C



the prefix form is:

+ A * B C



10.3 Postfix

Operator is written after operands.

A B +



For:

A + B * C



the postfix form is:

A B C * +



Postfix is also called **Reverse Polish Notation (RPN)**.

11. Why Expression Conversion is Required

Computers can evaluate expressions more easily when the order of operations is explicitly represented.

For example:

A + B * C



Multiplication has higher precedence than addition.

Therefore:

A + (B * C)



Postfix representation:

A B C * +



There is no need for parentheses or precedence rules during evaluation.

12. Operator Precedence

When converting expressions, operators have different priorities.

Operator	Meaning	Precedence
$\wedge$	Power	Highest
$*$ / $\%$	Multiplication, Division, Modulus	High
$+$ -	Addition, Subtraction	Low

For example:

A + B * C



* has higher precedence than + .

Therefore:

A B C * +



13. Associativity

Associativity determines the order of evaluation when operators have the same precedence.

Left Associative

+ - * / %



Example:

A - B + C



is evaluated as:

(A - B) + C



Right Associative

Exponentiation `^` is generally treated as right-associative.

`A ^ B ^ C`



means:

`A ^ (B ^ C)`



14. Infix to Postfix Conversion

A **stack** is used to convert an infix expression into postfix.

Rules

Scan the expression from left to right.

Rule 1 — Operand

If the symbol is an operand:

| Add it directly to the output.

Rule 2 — Left Parenthesis

If symbol is `(`:

| Push it onto the stack.

Rule 3 — Right Parenthesis

If symbol is `)`:

| Pop operators from the stack until `(` is found.

Discard the parentheses.

Rule 4 — Operator

If the symbol is an operator:

- Pop operators having higher or equal precedence from the stack.
- Then push the current operator.

Rule 5 — End of expression

After scanning the complete expression:

| Pop all remaining operators from the stack.

15. Algorithm: Infix to Postfix

INFIX_TO_POSTFIX(expression)



1. Create an empty stack.
2. Create an empty output.
3. Scan the expression from left to right.
4. If the symbol is an operand:
Add it to output.
5. Else if symbol is '(':
Push it onto stack.
6. Else if symbol is ')':
While TOP != '(':
Pop stack and add to output.
Pop '('.
7. Else if symbol is an operator:
While stack is not empty AND
precedence(TOP) >= precedence(symbol):
Pop stack and add to output.

Push symbol onto stack.
8. After scanning:
While stack is not empty:
Pop stack and add to output.
9. Return output.

16. Example: Infix to Postfix

Convert:

A + B * C



Step-by-step

Scan	Stack	Output
A	Empty	A
+	+	A
B	+	AB

Scan	Stack	Output
*	+ *	AB
C	+ *	ABC
End	Empty	ABC*+

Therefore:

Infix: A + B * C
Postfix: A B C * +



17. Another Example

Convert:

(A + B) * C



Steps

Symbol	Stack	Output
(	(	
A	(	A
+	(+	A
B	(+	AB
)	Empty	AB+
*	*	AB+
C	*	AB+C
End	Empty	AB+C*

Therefore:

Infix: (A+B)*C
Postfix: AB+C*



18. Complexity of Infix to Postfix

Suppose the expression contains n symbols.

Each symbol is:

- Scanned once.
- Pushed at most once.
- Popped at most once.

Therefore:

Time Complexity

$O(n)$



Space Complexity

$O(n)$



because the stack may contain up to n operators/parentheses.

19. Infix to Prefix Conversion

Another important expression conversion is:

Infix $\rightarrow$ Prefix



Example:

$A + B * C$



becomes:

$+ A * B C$



A common method is:

1. Reverse the infix expression.
2. Replace $($ with $)$ and $)$ with $($.
3. Convert the modified expression to postfix.
4. Reverse the resulting postfix expression.

Complexity

For n symbols:

- Time: $O(n)$

- Space: $O(n)$

20. Postfix Expression Evaluation

A stack can also be used to evaluate a postfix expression.

Consider:

2 3 + 4 *



This represents:

(2 + 3) * 4



Result:

20



21. Algorithm: Postfix Evaluation

Scan the postfix expression from left to right.

Rules

If operand:

Push operand onto stack.



If operator:

1. Pop the second operand.
2. Pop the first operand.
3. Perform the operation.
4. Push the result back.

Important

For subtraction and division, order is important.

If:

a b -



then:

a - b



not:

b - a



Algorithm

EVALUATE_POSTFIX(expression)



1. Create an empty stack.
2. Scan expression from left to right.
3. If symbol is an operand:
Push symbol onto stack.
4. If symbol is an operator:
operand2 = Pop()
operand1 = Pop()

result = operand1 operator operand2

Push(result)
5. After scanning all symbols:
Return Pop()

22. Example: Postfix Evaluation

Evaluate:

2 3 + 4 *



Step-by-step

Symbol	Operation	Stack
2	Push	2
3	Push	2, 3
+	2 + 3 = 5	5
4	Push	5, 4

Symbol	Operation	Stack
*	$5 \times 4 = 20$	20

Final result:

20



23. Another Postfix Example

Evaluate:

5 6 2 + * 12 4 / -



Equivalent expression:

$5 * (6 + 2) - (12 / 4)$



Calculation:

$6 + 2 = 8$



$5 * 8 = 40$

$12 / 4 = 3$

$40 - 3 = 37$

Therefore:

Answer = 37



24. Complexity of Postfix Evaluation

For n symbols:

Each symbol is scanned once.

Each operand is pushed once and each operator performs a constant number of stack operations.

Therefore:

Time Complexity

$O(n)$



Space Complexity

$O(n)$



25. Prefix Expression Evaluation

Prefix expressions can also be evaluated using a stack.

Example:

`+ 2 * 3 4`



represents:

`2 + (3 * 4)`



Result:

`14`



Algorithm

Scan the expression **from right to left**.

```
If operand:  
    Push it.  
  
If operator:  
    operand1 = Pop()  
    operand2 = Pop()  
    result = operand1 operator operand2  
    Push result
```



At the end:

`Pop() → final result`



Complexity

For `n` symbols:

- Time = $O(n)$
 - Space = $O(n)$
-

26. Stack Applications — Summary

Application	Use of Stack
Function calls	Stores activation records
Recursion	Maintains recursive calls
Undo operation	Stores previous states
Browser Back	Stores visited pages
Parentheses matching	Matches opening/closing brackets
Infix → Postfix	Stores operators
Postfix evaluation	Stores operands/results
Prefix evaluation	Stores operands/results
DFS	Stores unexplored vertices
Backtracking	Stores previous states

27. Introduction to Queue

A **Queue** is a linear data structure in which:

- Insertion occurs at the **REAR**
- Deletion occurs at the **FRONT**

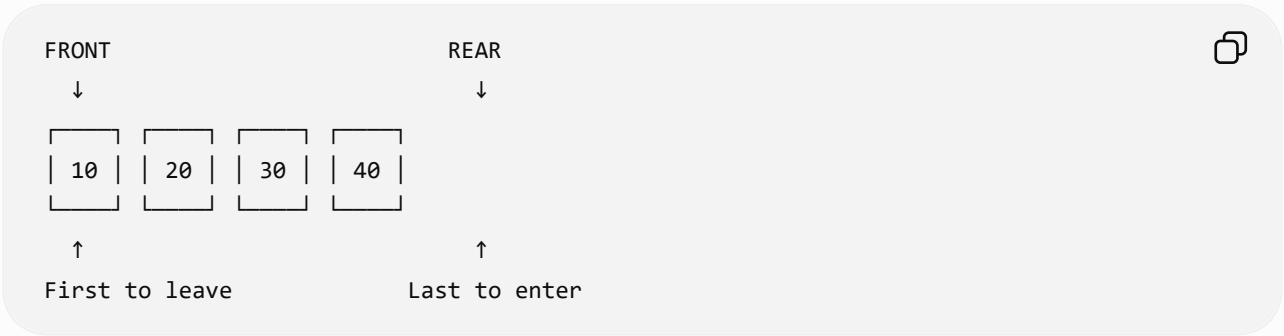
A queue follows:

FIFO — First In, First Out

The element inserted first is removed first.

Real-life example

A queue of people waiting at a counter:



28. Stack vs Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion	TOP	REAR
Deletion	TOP	FRONT
Main insertion operation	Push	Enqueue
Main deletion operation	Pop	Dequeue
Example	Stack of plates	Waiting line

29. Queue Operations

The major queue operations are:

1. Enqueue

Adds an element at the rear.

ENQUEUE (50)



2. Dequeue

Removes an element from the front.

3. Front / Peek

Returns the front element without removing it.

4. isEmpty

Checks whether queue is empty.

5. isFull

Checks whether queue is full in an array implementation.

30. Queue Example

Initially:

Queue :
[10, 20, 30]



↑ ↑
Front Rear

After:

ENQUEUE(40)



we get:

[10, 20, 30, 40]
↑ ↑
Front Rear



After:

DEQUEUE()



10 is removed:

[20, 30, 40]
↑ ↑
Front Rear



31. Queue Applications

Queues are used in:

- CPU scheduling
- Printer scheduling
- Network packet management
- Keyboard buffers
- Breadth First Search (BFS)
- Operating system processes
- Customer service systems
- Request handling in servers
- Data streaming
- Simulation systems

32. Important Complexity Summary

Operation / Algorithm	Time Complexity	Space Complexity
Stack Push	$O(1)$	$O(1)$

Operation / Algorithm	Time Complexity	Space Complexity
Stack Pop	$O(1)$	$O(1)$
Stack Peek	$O(1)$	$O(1)$
Stack isEmpty	$O(1)$	$O(1)$
Queue Enqueue*	$O(1)$	$O(1)$
Queue Dequeue*	$O(1)$	$O(1)$
Infix $\rightarrow$ Postfix	$O(n)$	$O(n)$
Infix $\rightarrow$ Prefix	$O(n)$	$O(n)$
Postfix Evaluation	$O(n)$	$O(n)$
Prefix Evaluation	$O(n)$	$O(n)$

*For a properly implemented array/circular queue or linked queue.

33. Important Exam Points

Stack

Stack = LIFO

Insertion and deletion occur at:

TOP

Operations:

Push
Pop
Peek
isEmpty
isFull



Basic operations:

$O(1)$

Queue

Queue = FIFO

Insertion occurs at:

REAR

Deletion occurs at:

FRONT

Operations:

Enqueue
Dequeue
Peek/Front
isEmpty
isFull



Expression Conversion

Most important rules:

Operand → Output

(→ Push

) → Pop until (

Operator → Pop higher/equal precedence operators,
then push current operator

End → Pop remaining operators



Complexity

For an expression containing n symbols:

Infix → Postfix = $O(n)$
Postfix Evaluation = $O(n)$
Infix → Prefix = $O(n)$
Prefix Evaluation = $O(n)$



Key idea: Stack operations are constant time, but expression algorithms take $O(n)$ because the entire expression must be scanned.



